E85: Digital Electronics and Computer Engineering

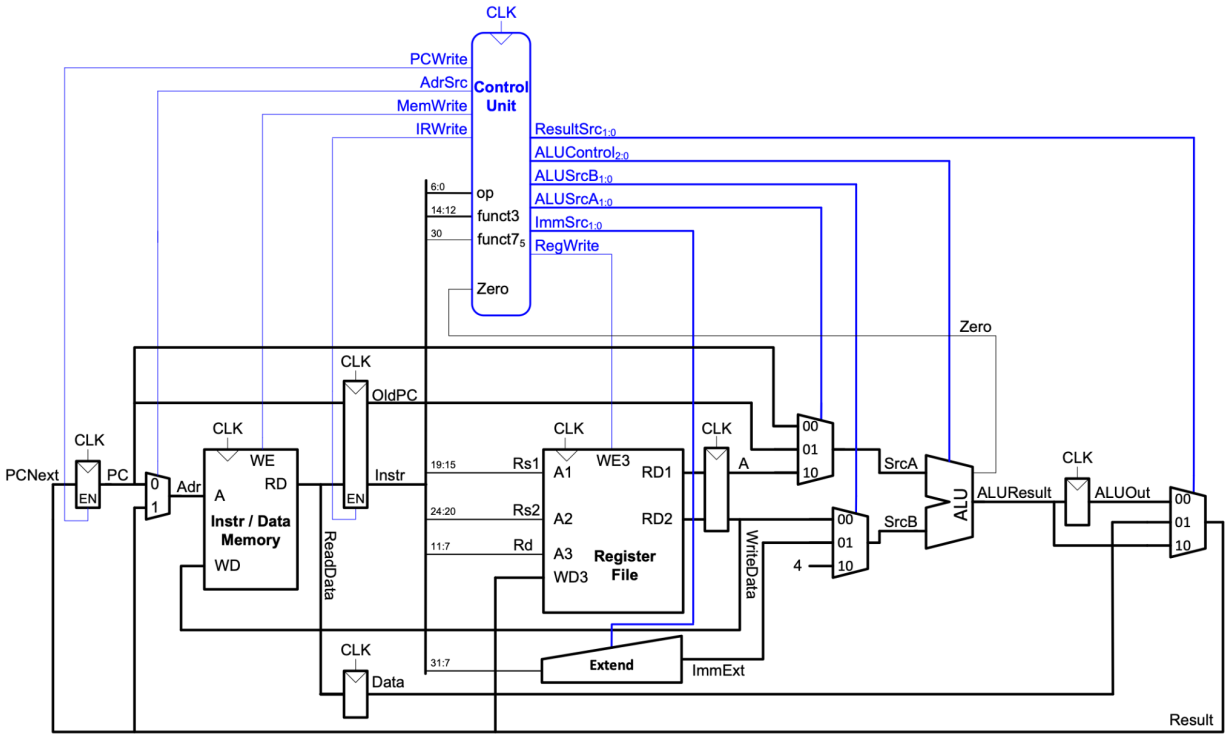
Lab 11: Multicycle Processor

This Google Doc, linked from Canvas, is the authoritative specification for the assignment.
We reserve the right to update it with hints/clarifications/corrections at any time if people ask good questions during the week.

This lab cannot be skipped or turned in late.

Objective

In this lab, you will build, simulate, and debug a multicycle RISC-V processor in SystemVerilog, as described in the textbook:

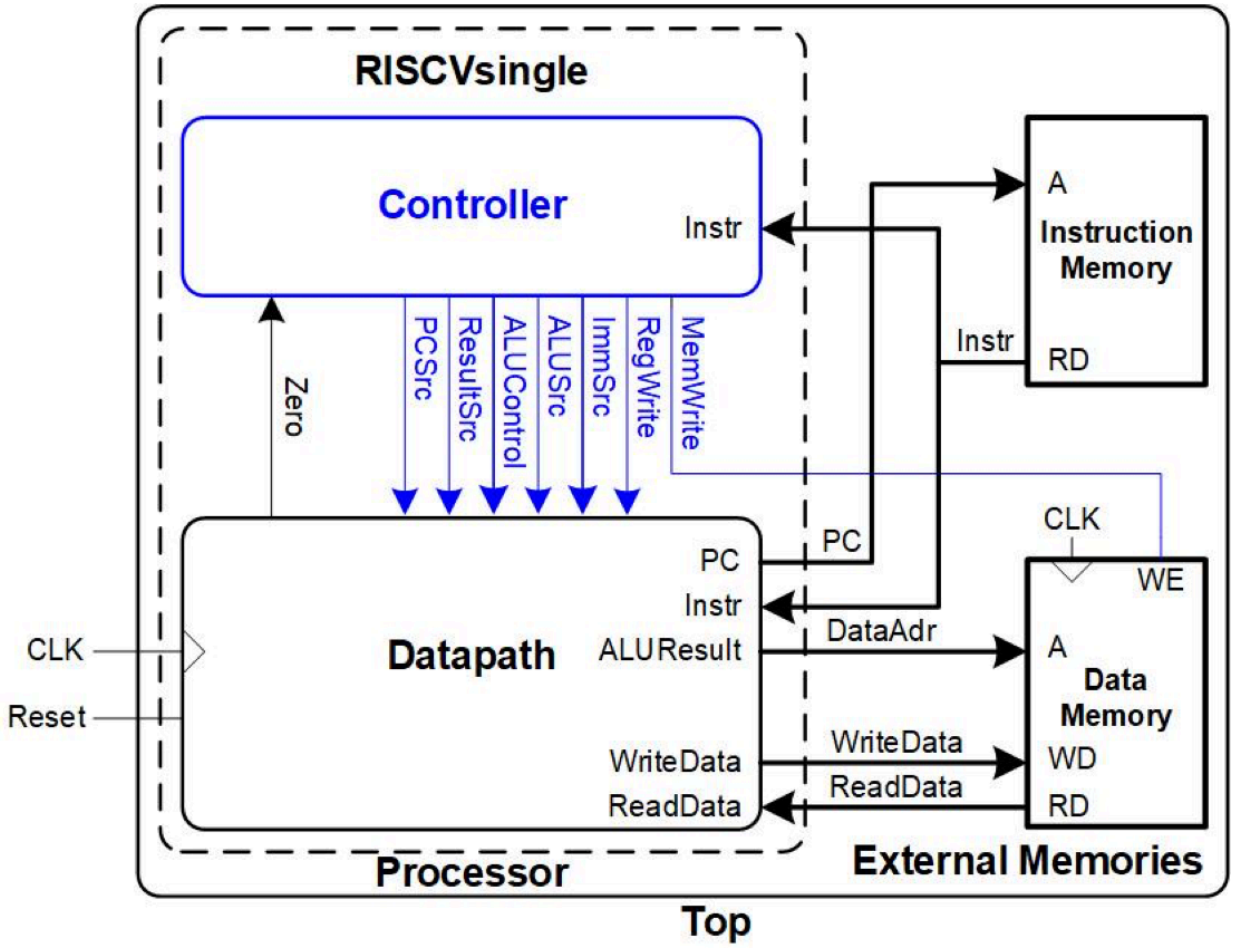


1. Warmup

Below is the high-level hierarchy of the *single-cycle* processor, showing the connections between the controller, datapath, instruction memory, and data memory.

Your multi-cycle processor has only a single unified memory and has slightly different control signals, so you will need to modify these connections.

- Sketch a similar diagram showing your controller, datapath, and memory module(s).
- Draw a box corresponding to your `riscv` processor that includes the controller and datapath.
- Label the signals passing between blocks.



2. Adding a Controller and Memory in SystemVerilog

Write a hierarchical SystemVerilog description of this system.

- By hierarchical, we mean that you should use separate Verilog modules for each box in the design (e.g., an outermost `top` module containing a `riscv` processor module and a memory module, where `riscv` includes a datapath and a control unit).

- You may re-use (copy) and adapt code from the following references:

- Your controller from Lab 10 (and modules from any previous labs)
- Verilog modules from the textbook (e.g., the building blocks in Chapter 4)
- **Any part of the complete code for the single-cycle processor,** which is available as RISCVsingle.sv and in Section 7.6 of the textbook.

- Your outermost module should have the following interface:

```
module top(input  logic        clk, reset,
           output logic [31:0] WriteData, DataAdr,
           output logic        MemWrite);


endmodule
```

Note: the outputs might surprise you because conceptually the `WriteData`, `DataAdr`, and `MemWrite` signals are purely internal to `top` (i.e., they are the wires connecting the `riscv` processor to memory).

For this lab, these internal signals have been *tapped out* (i.e., have extra connections to the outside world), since that makes your module easier to test.

2. Test Bench

The following files are available for testing:

- riscv_testbench.sv does the testing
- memfile.asm is the assembly language corresponding to the machine code
- memfile.dat (the machine code you'll use for testing)

Study the test bench to understand how it determines if your tests succeeded or failed.

3. Test Setup

1. Add the following line to your memory (where `RAM` is the name of your array of bytes) so that it loads the machine code when the simulation starts:

```
initial $readmemh("memfile.dat", RAM);
```

2. Complete the following table to predict what the processor should do while executing the first *three* instructions. (Only fill in the notes column if it is helpful to you to understand what is happening and why.)

Expected Operation (*after two cycles of reset*)

Step	PC	Instr	State	Result	Result Notes
3	00	00500113	S0: Fetch	4	PC+4
4	04	""	S1: Decode	X	OldPC+Immediate
5	04	""	S8: ExecuteI	X	ALUResult = x0 (0) + 5 = 5
6	04	""	S7: ALUWB	5	Result = ALUOUT
7	04	""	S0: Fetch	8	PC+4
8	08	00c00193	S1: Decode	X	OldPC+Immediate
9					
10					
11					
12					
13					
14					
15					
16					
17					
18					
19					
20					
21					
22					
23					
24					
25					
26					
27					
28					

29					
30					
31					
32					
33					
34					
35					
36					
37					
38					
39					
40					
41					
42					
43					

4. Testing and Debugging

Run the simulation to generate simulation waveforms at least for `clk`, `reset`, `PC`, `Instr`, `state`, `SrcA`, `SrcB`, `ALUResult`, `Adr`, `WriteData`, and `MemWrite`. Display the 32-bit signals in hexadecimal for ease of reading (select the signals and right click, then choose Radix). You may wish to add other signals to help debug.

Compare against your expectations. Fix any problems you may find until your code executes the program as expected and the testbench reports success.

Refer to the previous lab for debugging hints. Fix all relevant warnings from Questa (and Quartus, if you want to see what it says)

It will save you much time to carefully predict what each of the signals in your waveforms should be doing on each cycle, and to systematically debug beginning with the first discrepancy and working your way backward until you find a module with good inputs and bad outputs and have isolated the bug.

Common bugs include:

- Copying the *single-cycle* processor `top`, `riscv`, or `datapath` interface with signals that don't match the multi-cycle processor. Be sure you have a clear idea what belongs in each module, and consult the system

picture you drew above.

- Listing signals in different orders when you *define* the `module` vs. when you *use* the `module`.
- Forgetting to declare internal signals, or giving them the wrong widths.
- Inconsistent capitalization or spelling.

If you've checked these and your processor still isn't working, try adding all the outputs of the controller to your simulation and make sure none are floating or X.

If you still haven't found the problem, refer to your predicted waveforms in Table 1 and check that the processor is doing the right thing on each step. If the first few instructions are correct, you may need to extend the table to predict what the rest of the program should be doing. (Once you've filled out the table for several instructions, you may get the hang of the pattern and only fill out entries that are interesting…)

What to Turn In

1. Please indicate how many hours you spent on this lab. This will be helpful for calibrating the workload for next time the course is taught. 15 hours
2. The diagram showing your memory, `riscv`, datapath, and controller block hierarchy and the names of all signals between them.
3. Hierarchical SystemVerilog for your top-level processor module (and submodules).

   Please do not submit screenshots of code; instead, copy and paste the text into your answer document, and fiddle with the font, size, and color to make sure it's easily readable and the indentation makes sense.

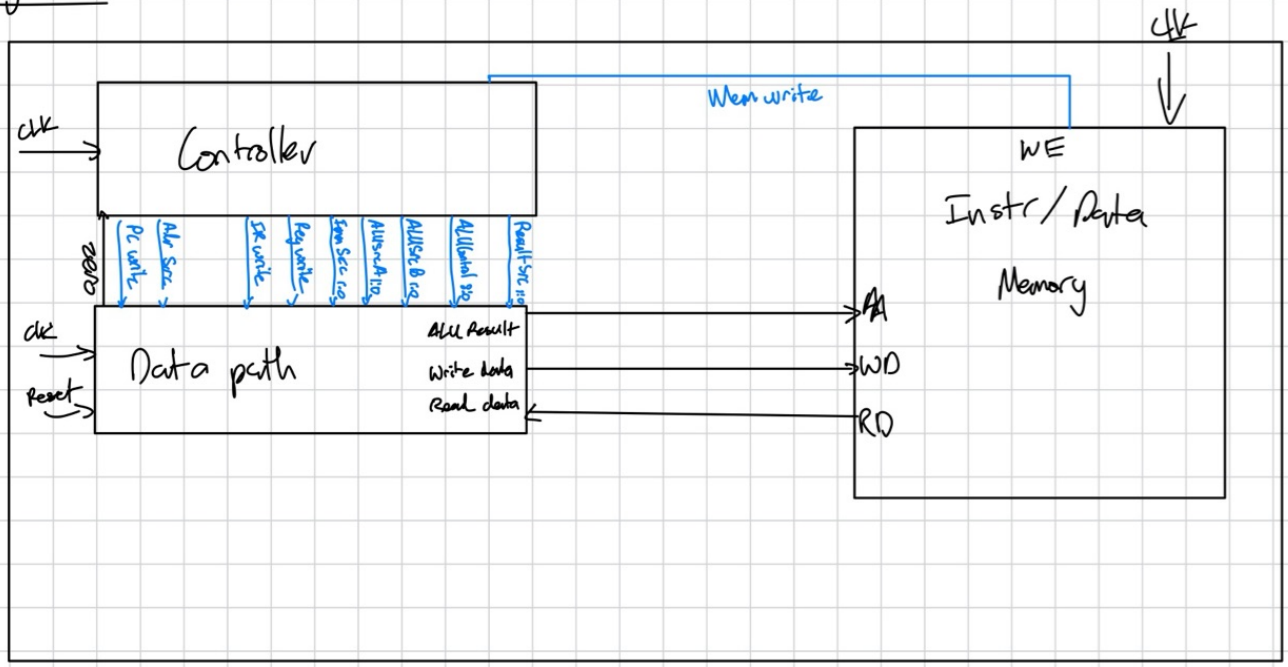
   You do not have to include the testbench or the memfiles.
4. Table 1 filled out for at least the first three instructions.
5. Simulation waveforms **<u>in the order listed above</u>** at least for the specified signals

   <u>And</u>, does your system pass your testbench?

Expected Operation (*after two cycles of reset*)

Step	PC	Instr	State	Result	Result Notes
3	00	00500113	S0: Fetch	4	PC+4
4	04	""	S1: Decode	X	OldPC+Immediate
5	04	""	S8: ExecuteI	X	ALUResult = x0 (0) + 5 = 5
6	04	""	S7: ALUWB	5	Result = ALUOUT
7	04	""	S0: Fetch	8	PC+4
8	08	00c00193	S1: Decode	X	OldPC+Immediate
9	08	"	S8: Execute I	x	
10	08	"	S7: ALUWB	12	
11	08	"	S0: Fetch	12	
12	12	ff7183 93	S1: Decode	x	
13	12	"	S8: Execute	x	
14	12	"	S7: ALUWB	3	
15	12	"	S0: Fetch	16	
16	16	0023e233	S1: Decode	x	
17	16	"	S6: Execute R	x	
18	16	"	S7: ALUWB	9	
19	16	"	S0: Fetch	20	
20					
21					
22					
23					
24					
25					
26					
27					
28					

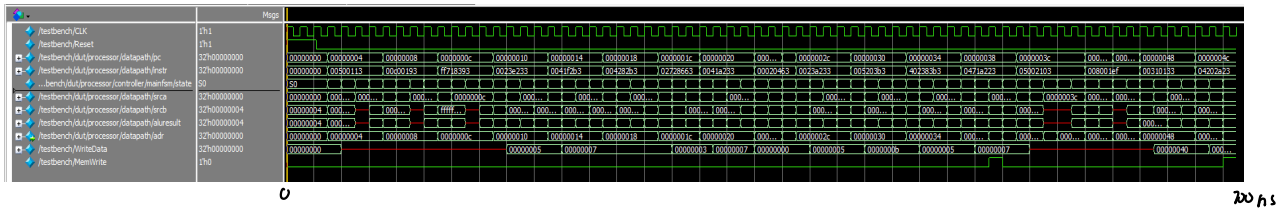
Diagram



And, circle or highlight the waves showing that the correct final value is written to the correct address, and make sure it is legible.

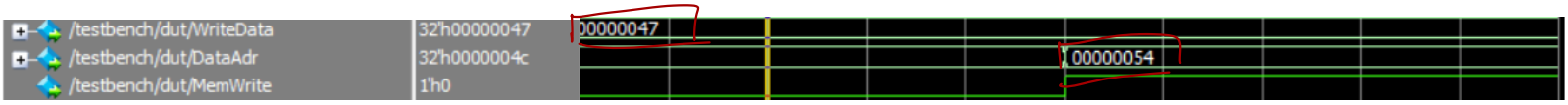
Please indicate any bugs you found in this lab manual, or any suggestions you would have to improve the lab.

All wave forms



```
VSIM 41> run 1000
# Simulation succeeded
# ** Note: $stop    : C:/Lab11_Folder/riscv_testbench.sv(30)
#    Time: 685 ns  Iteration: 1  Instance: /testbench
# Break in Module testbench at C:/Lab11_Folder/riscv_testbench.sv line 30
```

The system passed the test bench!



When MemWrite = 1, Write data = 47 and Data address = 54,
This matches our test bench values of Data address = 84 and Write data = 71
because 84 in Hexadecimal is 00000054 and 71 is 00000047
therefore the values and addresses are correct.

```
module flopr #(parameter WIDTH = 8)
(input logic clk, reset,
 input logic [WIDTH-1:0] d,
 output logic [WIDTH-1:0] q);
```

```
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else q <= d;
endmodule

module flopr2 #(parameter WIDTH = 8)
(input logic clk, reset, enable,
 input logic [WIDTH-1:0] d,
 output logic [WIDTH-1:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else if (enable) q <= d;
endmodule

module mux2 #(parameter WIDTH = 8)
(input logic [WIDTH-1:0] d0, d1,
 input logic s,
 output logic [WIDTH-1:0] y);
    assign y = s ? d1 : d0;
endmodule

module mux3 #(parameter WIDTH = 8)
(input logic [WIDTH-1:0] d0, d1, d2,
 input logic [1:0] s,
 output logic [WIDTH-1:0] y);
```

```
    assign y = s[1] ? d2 : (s[0] ? d1 : d0);
endmodule

module alu(input logic [31:0] a, b,
           input logic [2:0] alucontrol,
           output logic [31:0] result,
           output logic zero);
    logic [31:0] ib, sum;
    logic sub;
    assign sub = (alucontrol[1:0] == 2'b01);
    assign ib  = sub ? ~b : b;
    assign sum = a + ib + sub;
    always_comb
        case (alucontrol)
            3'b000: result = sum;
            3'b001: result = sum;
            3'b010: result = a & b;
            3'b011: result = a | b;
            3'b101: result = sum[31];
            default: result = 0;
        endcase
    assign zero = (result == 0);
endmodule

module RegFile(input logic clk, WE3,
```

```
            input logic [4:0] A1, A2, A3,
            input logic [31:0] WD3,
            output logic [31:0] RD1, RD2);
    logic [31:0] rf[31:0];
    always_ff @(posedge clk)
      if (WE3 && A3 != 0) rf[A3] <= WD3;
    assign RD1 = (A1 != 0) ? rf[A1] : 0;
    assign RD2 = (A2 != 0) ? rf[A2] : 0;
endmodule

module Extend(input logic [31:7] Instr,
            input logic [1:0] ImmSrc,
            output logic [31:0] ImmExt);
    always_comb
      case (ImmSrc)
        2'b00: ImmExt = {{20{Instr[31]}}, Instr[31:20]};
        2'b01: ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]};
        2'b10: ImmExt = {{20{Instr[31]}}, Instr[7], Instr[30:25],
                        Instr[11:8], 1'b0};
        2'b11: ImmExt = {{12{Instr[31]}}, Instr[19:12], Instr[20],
                        Instr[30:21], 1'b0};
        default: ImmExt = 32'bx;
      endcase
endmodule
```

```
module aluDecoder(input logic [2:0] funct3,
                  input logic f7b5,
                  input logic [1:0] aluop,
                  input logic op5,
                  output logic [2:0] alucontrol);
   logic kind;
   assign kind = f7b5 & op5;
   always_comb
      case (aluop)
         2'b00: alucontrol = 3'b000;
         2'b01: alucontrol = 3'b001;
         default:
            case ({kind, funct3})
               4'b0000: alucontrol = 3'b000;
               4'b1000: alucontrol = 3'b001;
               4'b0111: alucontrol = 3'b010;
               4'b0110: alucontrol = 3'b011;
               4'b0010: alucontrol = 3'b101;
               default: alucontrol = 3'bxxx;
            endcase
      endcase
endmodule

module instructionDecoder(input logic [6:0] op,
                          output logic [1:0] immSrc);
```

```
    always_comb
        case (op)
            7'b0100011: immSrc = 2'b01;
            7'b1100011: immSrc = 2'b10;
            7'b1101111: immSrc = 2'b11;
            default: immSrc = 2'b00;
        endcase
endmodule

module mainFSM(
    input logic [6:0] op,
    input logic clk, reset, zero,
    output logic pcWrite, regWrite, memWrite,
    output logic irWrite, adrSrc,
    output logic [1:0] resultSource,
    output logic [1:0] aluSourceA,
    output logic [1:0] aluSourceB,
    output logic [1:0] aluOp);

    typedef enum logic[3:0] {S0,S1,S2,S3,S4,S5,S6,S7,S8,S9,S10} state_t;
    state_t state, next;
    logic branch, pcUpd;

    assign pcWrite = (branch & zero) | pcUpd;
```

```
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else state <= next;

    always_comb begin
        regWrite = 0; memWrite = 0; irWrite = 0; adrSrc = 0;
        resultSource = 0; aluSourceA = 0; aluSourceB = 0;
        aluOp = 0; pcUpd = 0; branch = 0; next = S0;

        case (state)
            S0: begin irWrite=1; aluSourceB=2'b10; resultSource=2'b10; pcUpd=1;
next=S1; end
            S1: begin aluSourceA=2'b01; aluSourceB=2'b01;
                case(op)
                    7'b0000011,7'b0100011: next=S2;
                    7'b0110011: next=S6;
                    7'b0010011: next=S8;
                    7'b1101111: next=S9;
                    7'b1100011: next=S10;
                endcase
            end
            S2: begin aluSourceA=2'b10; aluSourceB=2'b01;
next=(op==7'b0000011)?S3:S5; end
            S3: begin adrSrc=1; next=S4; end
            S4: begin regWrite=1; resultSource=2'b01; next=S0; end
            S5: begin adrSrc=1; memWrite=1; next=S0; end
```

```
        S6: begin aluSourceA=2'b10; aluOp=2'b10; next=S7; end
        S7: begin regWrite=1; next=S0; end
        S8: begin aluSourceA=2'b10; aluSourceB=2'b01; aluOp=2'b10;
next=S7; end
        S9: begin aluSourceA=2'b01; aluSourceB=2'b10; pcUpd=1; next=S7;
end
        S10: begin aluSourceA=2'b10; aluOp=2'b01; branch=1; next=S0; end
      endcase
   end
endmodule

module controller(
   input logic clk, reset,
   input logic [6:0] op,
   input logic [2:0] funct3,
   input logic funct7b5,
   input logic zero,
   output logic [1:0] immsrc,
   output logic [1:0] alusrcA, alusrcB,
   output logic [1:0] resultsrc,
   output logic adrsrc,
   output logic [2:0] alucontrol,
   output logic irwrite, pcwrite,
   output logic regwrite, memwrite);
   logic [1:0] aluOp;
```

```
    instructionDecoder ID(op, immsrc);
    mainFSM FSM(op, clk, reset, zero,
                pcwrite, regwrite, memwrite,
                irwrite, adrsrc,
                resultsrc, alusrcA, alusrcB, aluOp);
    aluDecoder ALD(funct3, funct7b5, aluOp, op[5], alucontrol);
endmodule

module datapath(
    input logic clk, reset,
    input logic [1:0] immsrc,
    input logic [1:0] alusrcA, alusrcB,
    input logic [1:0] resultsrc,
    input logic adrsrc,
    input logic [2:0] alucontrol,
    input logic irwrite, pcwrite,
    input logic regwrite,
    input logic [31:0] readdata,
    output logic [6:0] op,
    output logic [2:0] funct3,
    output logic funct7b5,
    output logic zero,
    output logic [31:0] adr, writedata);

    logic [31:0] pc, oldpc, instr, data;
```

```
logic [31:0] ra, rb, a, imm, sa, sb;
logic [31:0] aluR, aluOut, res;

flopr2 #(32) PC(clk, reset, pcwrite, res, pc);
mux2  #(32) AM(pc, res, adrsrc, adr);
flopr2 #(32) OPC(clk, reset, irwrite, pc, oldpc);
flopr2 #(32) IR(clk, reset, irwrite, readdata, instr);
flopr  #(32) DR(clk, reset, readdata, data);

assign op      = instr[6:0];
assign funct3  = instr[14:12];
assign funct7b5 = instr[30];

RegFile RF(clk, regwrite, instr[19:15], instr[24:20], instr[11:7], res, ra, rb);
Extend EX(instr[31:7], immsrc, imm);
flopr #(32) AR(clk, reset, ra, a);
flopr #(32) BR(clk, reset, rb, writedata);

mux3 #(32) SA(pc, oldpc, a, alusrcA, sa);
mux3 #(32) SB(writedata, imm, 32'd4, alusrcB, sb);

alu U(sa, sb, alucontrol, aluR, zero);
flopr #(32) AO(clk, reset, aluR, aluOut);

mux3 #(32) RM(aluOut, data, aluR, resultsrc, res);
```

```
endmodule

module memory(
    input logic clk, WE,
    input logic [31:0] A, WD,
    output logic [31:0] RD);
    logic [31:0] RAM[63:0];
    initial $readmemh("memfile.dat", RAM);
    assign RD = RAM[A[31:2]];
    always_ff @(posedge clk)
        if (WE) RAM[A[31:2]] <= WD;
endmodule

module risc_Processor(
    input logic clk, reset,
    input logic [31:0] readdata,
    output logic memwrite,
    output logic [31:0] adr, writedata);
    logic pcw, as, is, rw;
    logic [1:0] rs, aA, aB, ims;
    logic [2:0] ac;
    logic [6:0] op;
    logic [2:0] f3;
    logic f7, z;
```

```
    datapath DP(clk, reset, ims, aA, aB, rs,
                as, ac, is, pcw, rw,
                readdata, op, f3, f7, z,
                adr, writedata);

    controller CT(clk, reset, op, f3, f7, z,
                  ims, aA, aB, rs,
                  as, ac, is, pcw, rw, memwrite);
endmodule

module top(
    input logic clk, reset,
    output logic [31:0] WriteData, DataAdr,
    output logic MemWrite);
    logic [31:0] ReadData;
    risc_Processor CPU(clk, reset, ReadData, MemWrite, DataAdr, WriteData);
    memory DM(clk, MemWrite, DataAdr, WriteData, ReadData);
endmodule
```